arXiv:2503.04346v1 [cs.CL] 6 Mar 2025

Adding Alignment Control to Language Models

Wenhong Zhu [1 2] **Weinan Zhang** [1 2] **Rui Wang** [1]

Abstract

Post-training alignment has increasingly become a crucial factor in enhancing the usability of language models (LMs). However, the strength of alignment varies depending on individual preferences. This paper proposes a method to incorporate alignment control into a single model, referred to as *CLM*. This approach adds one identity layer preceding the initial layers and performs preference learning only on this layer to map unaligned input token embeddings into the aligned space. Experimental results demonstrate that this efficient fine-tuning method performs comparable to full fine-tuning. During inference, the input embeddings are processed through the aligned and unaligned layers, which are then merged through the interpolation coefficient. By controlling this parameter, the alignment exhibits a clear interpolation and extrapolation phenomenon.

1. Introduction

One of the key successes of Large Language Models (LLMs) is their ability to align closely with human values and preferences (Ouyang et al., 2022). These models typically hinge on a series of critical training phases (OpenAI, 2024). First, they undergo pre-training on vast corpora to master the ability to predict the next token (Radford et al., 2019). Next, the pre-trained models are fine-tuned through supervised fine-tuning (SFT) to better adapt to specific domains (Wei et al., 2021; Yang et al., 2024b). Finally, preference-based optimization methods are employed, such as Reinforcement Learning from Human Feedback (RLHF) (Christiano et al., 2017) and Direct Preference Optimization (DPO) (Rafailov et al., 2024). These approaches ensure the model avoids engaging in factual inaccuracies, exhibiting biases, and displaying other undesirable behaviors (Bai et al., 2022).

Across these alignment approaches, the core objective remains consistent: to maximize the expected reward from an implicit or explicit reward function while incorporating the KL-divergence from the reference policy as a regularization term (Schulman et al., 2017; Gao et al., 2023; Rafailov et al., 2024). The strength of this regularization plays a pivotal role in determining the alignment outcome (Ziegler et al., 2019), as excessive regularization can overly constrain alignment, whereas insufficient regularization may lead to reward hacking (Pan et al., 2022).

Determining the optimal alignment strength often requires a process of trial and error (Meng et al., 2024). Moreover, the optimal alignment strength can vary based on cultural, ethical, and individual perspectives (Anwar et al., 2024). This variability makes it challenging to devise a universal approach for embedding ideal alignment into an LLM, as the definition of "optimal" may differ depending on who determines it. Recent studies have conceptualized RLHF/DPO models as interpolated outcomes between an initial SFT model and a hypothetically better-aligned model (Liu et al., 2024; Zheng et al., 2024a; Zhu et al., 2024). By introducing an interpolation coefficient, these works enable both the interpolation and extrapolation of alignment strength. This raises an intriguing question: *could this interpolation coefficient be integrated into a single model and exposed as a user-controllable interface—similar to a temperature parameter—allowing users to customize the alignment strength dynamically?*

In this work, we first adopt a distillation method to transfer extrapolated and interpolated alignment into the aligned model, thereby consolidating these observations (see Sec 3). While this approach improves inference efficiency, it comes at the cost of training efficiency. Our analysis reveals that the bottom layers of the LM play a crucial role in preference learning (see Sec 6.2). To construct an alignment-controllable language model (*CLM*), we duplicate the last layer and insert it as an identity layer preceding the original ones (see Sec 4.1). Preference learning is then confined to this newly added layer while all other layers remain frozen (see Sec 4.2). This design effectively maps unaligned input token embeddings into aligned spaces, ensuring the desired alignment effect (see Sec 6.3). As shown in Figure 1, the input embedding is processed through the aligned and unaligned layers during inference, with the output logits merged using an interpolation coefficient λ. This mecha-

[1]Department of Computer Science, Shanghai Jiao Tong Univeristy, Shanghai, China [2]Shanghai Innovation Institute, Shanghai, China. Correspondence to: Wenhong Zhu <zwhong714@sjtu.edu.cn>.

Preprint.

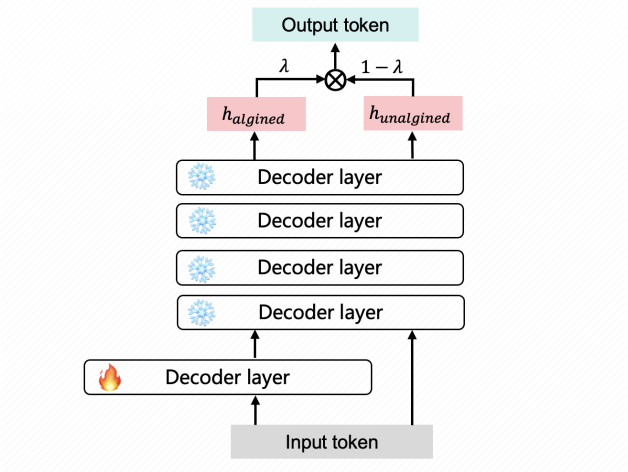

Figure 1. Inference architecture: The input embedding is fed simultaneously into the newly added aligned layer and the original first layer of the LM. The hidden states from both paths are propagated through all layers and merged after the LM head layer.

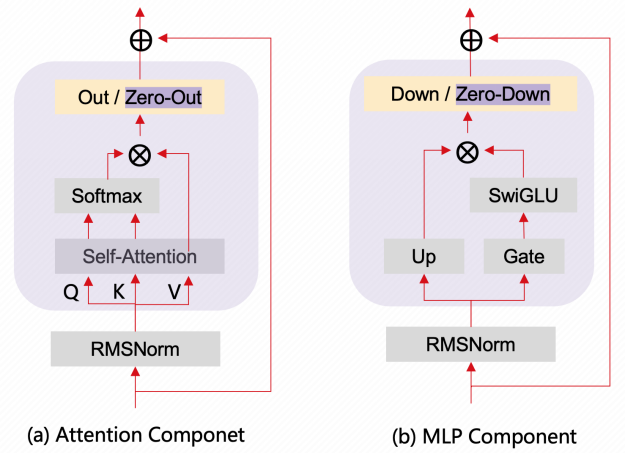

Figure 2. Overview of attention and MLP component. Identity copy makes the last projector of each component with weight and bias to zero.

nism enables precise alignment control. In summary, our contributions are as follows:

- We found that performing alignment in the bottom layers is more effective than in the upper layers, a result that has been scarcely explored in previous studies.
- We introduce *CLM*, which adds an identity layer preceding the original layers of LM and only performs preference learning on this layer. It is an efficient training method that achieves performance comparable to full fine-tuning.
- To our knowledge, *CLM* is the first to incorporate inference-time realignment considerations into the training process explicitly. The alignment strength manifests clear extrapolation and interpolation by appropriately controlling the interpolation coefficient λ.

2. Preliminary

2.1. Transformer Decoder Layer

The current mainstream LMs based on transformer architecture typically have multiple decoder layers $(\phi_0, \phi_1, ..., \phi_L)$ (OpenAI, 2024). Each layer consists of an attention component and an MLP component, as shown in Figure 2. Given an input h_{t-1}, each layer produces the output h_t as described by the following equations:

$$\begin{aligned} h'_{t-1} &= h_{t-1} + \text{Attention}(\text{RMSNorm}(h_{t-1})) \\ h_t &= h'_{t-1} + \text{MLP}\left(\text{RMSNorm}\left(h'_{t-1}\right)\right). \end{aligned} \tag{1}$$

Both components have a projector to ensure that the input and output dimensions of the module are consistent, which facilitates the combination with residual connection.

Identity copy The identity copy is defined as $\phi_{\text{id}}(h_{t-1}) = h_{t-1}$ which means the input and output are identical. This can be achieved as long as Attention(RMSNorm(h_{t-1})) = 0 and MLP(RMSNorm(h'_{t-1})) = 0. Then, the input is directly the result of the output due to the residual. We initialize the W_{out} and W_{down} weight matrices as marked in dark purple in Figure 2 in the identity layer to zero. As a result, the entire layer is reduced to an identity layer at initialization, preserving the output from the initial model.

2.2. Fine-tuning from Human Feedback

Autoregressive LM Given a query sequence $x := (x_1, \ldots, x_m) \in \mathcal{X}$, an auto-regressive LM defines a probability distribution over possible response sequences $y := (y_1, y_2, \ldots, y_n) \in \mathcal{Y}$. The probability $\pi_\theta(y \mid x)$ can be decomposed using the chain rule of probability as $\pi_\theta(y \mid x) = \prod_{t=1}^{n} \pi_\theta\left(y_t \mid y_{<t}, x\right)$, where $y_{<t}$ denotes $\{y_1, y_2, ..., y_{t-1}\}$.

Given a pre-trained and typically SFT reference model $\pi^{\text{ref}}(y \mid x)$, we need to use human-labeled datasets reflecting human preferences further for aligning LMs with human preferences (Wei et al., 2021). The alignment approaches such as PPO (Schulman et al., 2017) and DPO (Rafailov et al., 2024), whose optimization objective is generally to maximize the expected reward $r(x, y)$ from an implicit or explicit reward function while including a KL-divergence term from the reference policy as a penalty for divergence. The main objective is as follows:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{X}, y\sim\pi_\theta(y|x)} \left[r(x,y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi^{\text{ref}}(y \mid x)} \right], \quad (2)$$

where β is a regularization parameter. According to Rafailov et al. (2024), it has the closed solution form as follows:

$$\pi^*_\theta(\beta)(y \mid x) = \frac{\pi^{\text{ref}}(y \mid x) \exp\left[\frac{1}{\beta} r(x,y)\right]}{\sum_{y'} \pi^{\text{ref}}(y' \mid x) \exp\left[\frac{1}{\beta} r(x,y')\right]}. \quad (3)$$

Typically, we can transform the above equation by representing $r(x,y)$ as

$$\frac{1}{\beta} r(x,y) = \log \frac{\pi^*_\theta(\beta)(y \mid x)}{\pi^{\text{ref}}(y \mid x)} + \log Z(x), \quad (4)$$

where $Z(x) := \sum_{y'} \pi^{\text{ref}}(y' \mid x) \exp\left(\frac{1}{\beta} r(x,y')\right)$ is the partition function.

3. Alignment Interpolation And Extrapolation

As shown in Equation 2, the KL regularization parameter β encourages the policy model $\pi_\theta(y \mid x)$ to remain close to its initial state $\pi^{\text{ref}}(y \mid x)$ (Geist et al., 2019), with its value determining the strength of alignment. To adjust the alignment strength of the LM, one can modify the value of β, which can be achieved by scaling it with a factor λ during training. This adjustment leads to an updated optimal solution, expressed as $\pi^*_\theta(\beta/\lambda)(y \mid x)$ as follows:

$$\pi^*_\theta(\beta/\lambda)(y \mid x) = \frac{\pi^{\text{ref}}(y \mid x) \exp\left[\frac{\lambda}{\beta} r(x,y)\right]}{\sum_{y'} \pi^{\text{ref}}(y' \mid x) \exp\left[\frac{\lambda}{\beta} r(x,y')\right]}. \quad (5)$$

Equation 4 shows that the reward function can be expressed as the log-likelihood ratio between the policy and reference models. Substituting Equation 4 into Equation 5 yields the following equation:

$$\pi^*_\theta(\beta/\lambda)(y \mid x) = \frac{\pi^{\text{ref}}(y \mid x) \left[\frac{\pi^*_\theta(\beta)(y|x)}{\pi^{\text{ref}}(y|x)}\right]^\lambda}{\sum_{y'} \pi^{\text{ref}}(y' \mid x) \left[\frac{\pi^*_\theta(\beta)(y'|x)}{\pi^{\text{ref}}(y'|x)}\right]^\lambda}. \quad (6)$$

However, computing it is infeasible due to the normalization constant involving all possible sequences. Through the auto-regressive property of LMs, decoding realignment (DeRa) (Liu et al., 2024) demonstrates that Equation 6 can be approximated by per-token level. Specifically, it loads two models during the generation: a reference model and an aligned model. When decoding tokens step by step, it combines the logits from the reference model, $\boldsymbol{h}_t^{\text{ref}}$, with its from the β-regularization-aligned model, $\boldsymbol{h}_t^{\theta}(\beta)$, at each time step t, as described in Equation 7.

$$\widehat{\pi}_\theta(\beta/\lambda)(\cdot \mid x, y_{<t}) := \text{softmax}\left[\lambda \boldsymbol{h}_t^{\theta}(\beta) + (1-\lambda)\boldsymbol{h}_t^{\text{ref}}\right]. \quad (7)$$

The interpolation parameter λ functions as readjusting the alignment strength during the inference. It suggests that when $0 < \lambda < 1$, the realigned model can interpolate the reference model and aligned model, and an appropriate larger λ can perform an extrapolation between (Liu et al., 2024).

As previously mentioned, this process doubles both the decoding time and memory consumption. One practical approach to this issue is identifying an appropriate regularization strength and retraining the reference model. Alternatively, the distillation technique can be employed to realign the aligned model. It can also be used to verify the correctness of Equation 7 and demonstrate that the alignment strength can be both interpolated and extrapolated. To achieve this, we minimize the following objective function:

$$\mathcal{L}_{\text{KD}} = \sum_{t=1}^{|y|} D_{\text{KL}}(\pi_\theta(\cdot|x, y_{<t}) || \hat{\pi}_\theta(\beta/\lambda)(\cdot|x, y_{<t})). \quad (8)$$

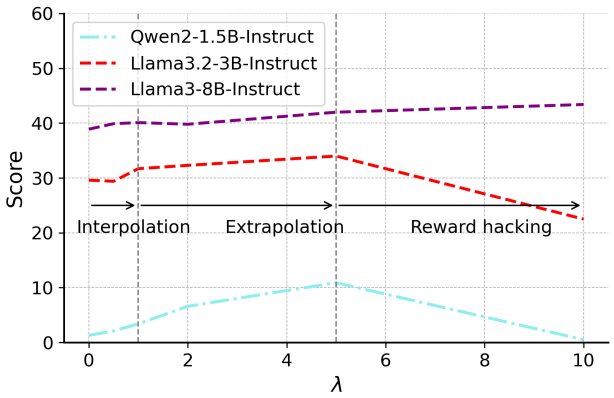

Figure 3. The win rate score on the Arena-Hard (Li et al., 2024b) benchmark varies with the interpolation parameter λ. When $\lambda = 1$, the model corresponds to a DPO-aligned (Rafailov et al., 2024) model trained using the Ultrafeedback (Cui et al., 2024) preference dataset for preference learning.

As illustrated in Figure 3, the results further highlight the role of the interpolation parameter λ. However, an inter-

esting question remains: how can we incorporate this realignment control capability into a single model without retraining or distillation? We hope this alignment control capability allows users to customize their preferred alignment strength, recognizing that these high-level values are inherently subjective and can be interpreted differently by individuals (Anwar et al., 2024).

4. CLM

4.1. Layer Expansion

Layer expansion involves inserting additional layers into the original layer structure. In this section, we aim to ensure that the added layers do not compromise the fundamental capabilities of LMs. This can be achieved by incorporating identity layers. Additionally, these layers are designed to map unaligned token embeddings to aligned spaces, effectively facilitating preference learning.

Unlike LLAMA-PRO (Wu et al., 2024), which partitions the transformer layers into groups and duplicates the top layer of each group, our approach takes a different strategy. As illustrated in Figure 4, we duplicate the first layer from the original model and insert it as an identical mapping, as described in Section 2. This method ensures that the original distribution predicted by the model remains unchanged.

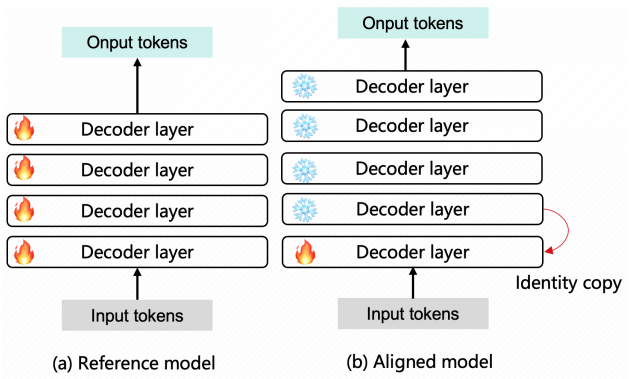

Figure 4. (a) Previous methods fine-tune all layers to facilitate preference learning. (b) In contrast, we perform preference learning exclusively on the added layer while keeping the original layers of the LM frozen.

4.2. Training

As shown in Figure 4, we freeze the original layers of the LMs and perform preference learning only on the added layer. The final LM head, which projects the hidden state to the token distribution, is also frozen. This training schema protects the originally predicted distribution and guarantees preference tuning starting from the original distribution.

Given a static dataset of comparisons $\mathcal{D} = \left\{x^{(i)}, y_w^{(i)}, y_l^{(i)}\right\}_{i=1}^{N}$, where y_w and y_l represent the preferred and dispreferred completions, respectively. We mainly use the DPO objective to perform preference learning only on the added layer as follows:

$$\mathcal{L}_{\text{DPO}}\left(\pi_\theta\right) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log\sigma\left(\beta\left(\log\frac{\pi_\theta\left(y_w \mid x\right)}{\pi^{\text{ref}}\left(y_w \mid x\right)} - \log\frac{\pi_\theta\left(y_l \mid x\right)}{\pi^{\text{ref}}\left(y_l \mid x\right)}\right)\right)\right]. \tag{9}$$

We aim for the added layer to map the unaligned embeddings into aligned spaces. The remaining frozen layers preserve the LM's language modeling capabilities and transform these embeddings into token distributions.

4.3. Inference

The decoding architecture is depicted in Figure 1. During the inference phase, the LM processes the input embeddings by passing them through the newly added aligned layer alongside the original first layer of the LM. The hidden states generated by both layers are retained and subsequently fed into the remaining layers. The aligned and unaligned logits are combined in the LM head layer using the interpolation parameter λ. This parameter functions similarly to a temperature setting, enabling users to customize the desired alignment strength. This architecture is fully compatible with the vLLM inference engine (Kwon et al., 2023).

5. Experiments

Models We implement our proposed method on the Llama3.2-3B (Dubey et al., 2024), Llama3.1-8B (Dubey et al., 2024), Qwen2.5-1.5B (Yang et al., 2024a) and Qwen2.5-7B models (Yang et al., 2024a). We aim to evaluate the method's effectiveness across diverse models and parameter scales. All models used in the experiments are selected from their respective base versions.

Datasets We train the base models on the UltraChat-200k (Ding et al., 2023) dataset that contains 1.5 million high-quality multi-turn dialogues and covers various topics and instructions to obtain the SFT models. Then, we perform alignment using DPO on the UltraFeedback dataset (Cui et al., 2024) based on SFT models.

Evaluation We evaluate our models primarily on three widely adopted, open-ended instruction following benchmarks: MT-Bench (Zheng et al., 2023), AlpacaEval 2 (Dubois et al., 2024), and Arena-Hard v0.1 (Li et al., 2024b). These benchmarks assess the models' conversational versatility across diverse queries and are commonly